实验六：生成对抗网络（GAN）

- 实验六：生成对抗网络（GAN）
  - 概述
    - 实现内容
  - 环境配置
  - GAN 的目标函数
  - 任务总览与推荐完成顺序
  - Task 1：潜变量与插值
    - 步骤 1.1：`sample_noise`
    - 步骤 1.2：`interpolate_latents`
    - 检查点 A
  - Task 2：DCGAN 网络
    - 步骤 2.1：`Generator`
    - 步骤 2.2：`Discriminator`
    - 步骤 2.3：`weights_init`
    - 检查点 B
  - Task 3：对抗损失
    - 步骤 3.1：`discriminator_loss`
    - 步骤 3.2：`generator_loss`
    - 检查点 C
  - Task 4：交替训练与梯度隔离
    - 步骤 4.1：`set_requires_grad`
    - 步骤 4.2：`discriminator_step`
    - 步骤 4.3：`generator_step`
    - 步骤 4.4：`train_one_epoch`
    - 检查点 D
  - Task 5：训练、可视化与诊断
    - 步骤 5.1：`batch_diversity`
    - 检查点 E：全部通过
    - 步骤 5.2：正式训练
    - 如何分析训练效果
  - 实验报告要求
  - 提供的文件
  - 评分标准
    - 第一部分：自动化测试
    - 第二部分：训练与报告
    - 提交要求
  - 参考资料

概述

生成对抗网络（Generative Adversarial Network, GAN）通过两个网络的对抗训练学习数据分布[1]：生成器（Generator, G）将随机潜变量 $z \sim \mathcal{N}(0, I)$ 映射为图像 $G(z)$；判别器（Discriminator, D）区分真实图像与生成图像。判别器努力分清真假，生成器则学习让生成图像被判为真。

本次实验的目标是：使用 PyTorch 搭建 DCGAN[2]，在 Fashion-MNIST 的 28×28 灰度服饰图像上训练生成模型[5]。重点是实现数值稳定的对抗损失、交替优化与梯度隔离，并结合固定噪声样本、多样性指标和潜变量插值分析训练效果。

实现内容

按依赖关系，依次完成以下五个部分：

顺序	部分	内容	所在文件
①	Task 1	噪声采样与潜变量插值	`src/training.py`、`src/metrics.py`
②	Task 2	DCGAN 生成器、判别器与权重初始化	`src/models.py`
③	Task 3	对抗损失与单边标签平滑	`src/losses.py`
④	Task 4	判别器更新 → 生成器更新 → 完整 epoch	`src/training.py`
⑤	Task 5	多样性诊断、正式训练与实验报告	`src/metrics.py`、`train_gan.py`

> 本实验使用 PyTorch 自动微分，无需手写反向传播。数据加载与命令行训练脚本已提供。

环境配置

请首先安装 `uv` Python 项目管理工具（已安装可跳过）：

```
# On macOS and Linux.
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# On Windows.
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

安装完成后，在学生版的项目根目录（即包含 `pyproject.toml` 的目录）下执行：

```
uv sync
```

项目通过 `.python-version` 固定使用 Python 3.11。该命令会在根目录下创建 `.venv` 虚拟环境，并安装 PyTorch、torchvision、NumPy、matplotlib 和 pytest。首次安装 PyTorch 需要下载较大的依赖包，请耐心等待。若使用 IDE，可将 Python 解释器设置为该虚拟环境中的解释器。

接着运行测试，检查环境是否配置成功：

```
uv run pytest
```

若输出类似以下内容，则说明环境配置成功：

```
FFFFFFFFFFFFFFFFFFFF.FFFFFFFFFFFF                                        [100%]
========================== short test summary info ===========================
FAILED tests/test_losses.py::test_discriminator_loss_matches_pytorch_definition - NotImplementedError
FAILED tests/test_losses.py::test_generator_loss_matches_non_saturating_objective - NotImplementedError
...
32 failed, 1 passed
```

其中，1 个通过的测试是 `test_models_reject_non_positive_dimensions`，只检查已经提供的模型构造参数校验。其余 32 个测试会调用尚未实现的函数，因此抛出 `NotImplementedError`，属于预期行为，不表示环境安装失败；完成对应任务后，它们才会逐一通过。不要在尚未编写代码时就期待看到全部 33 个测试通过。

> 本节展示的是学生版的初始状态。所有测试都使用 CPU 上的小型张量，不下载 Fashion-MNIST，也不需要 GPU。若出现 `ModuleNotFoundError` 或导入阶段的 `ERROR`，则不是上述预期失败，应先检查依赖和解释器设置。

调试时可按文件或测试名筛选；需要查看报错位置时，加上 `--tb=short`：

```
uv run pytest tests/test_models.py
uv run pytest tests/test_training.py -k sample_noise
uv run pytest tests/test_losses.py --tb=short
```

后续各步骤会给出单独的验证命令和应看到的通过数量。只有实现完该步骤及其依赖后，才应得到相应结果；使用 `-k` 时出现 `deselected`，表示其余测试被筛除，不是失败。

本文命令默认在项目根目录执行，测试文件均位于 `tests/`。也可以在 `tests/` 目录下测试，此时省略文件名前的 `tests/`；例如完成步骤 1.1 后，在该目录执行：

```
uv run pytest test_training.py -k sample_noise
```

同样应看到 5 passed。

训练脚本默认优先使用 CUDA，不可用时回退到 CPU，也可通过 `--device cpu` 或 `--device cuda` 显式指定。设备选择不影响本实验需要实现的函数接口。

GAN 的目标函数

原始 GAN 的极小极大目标为[1]：

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))].$$

这里的 $D(x)$ 表示样本为真的概率。代码中的判别器则输出未经 sigmoid 的分数，称为 logit。设真实图像与生成图像的 logit 分别为 s_r、s_f，判别器最小化：

$$L_D = \mathrm{BCELogits}(s_r, 1) + \mathrm{BCELogits}(s_f, 0).$$

生成器采用非饱和损失：

$$L_G = \mathrm{BCELogits}(s_f, 1).$$

即让生成图像被判为真。与直接最小化 $\log(1 - D(G(z)))$ 相比，非饱和损失在判别器能轻易识别假图的训练初期，仍能为生成器提供较强的梯度。

> `binary_cross_entropy_with_logits` 已将 sigmoid 与二元交叉熵合并，以保证数值稳定。判别器末尾不得添加 `Sigmoid`，计算损失前也不要手动调用 `sigmoid`。

任务总览与推荐完成顺序

建议按下表从上到下推进，每完成一组函数就运行对应测试，再进入下一步。步骤后的验证用于单独调试，检查点用于汇总验证；表中的测试数量均包含参数化用例。

步骤	函数或方法	依赖	完成后运行的验证	所在文件
1.1	`sample_noise`	无	5 个 `sample_noise` 测试	`src/training.py`

步骤	函数或方法	依赖	完成后运行的验证	所在文件
1.2	`interpolate_latents`	无	3 个 `interpolation` 测试	`src/metrics.py`
1	检查点 A： 潜变量工具通过	1.1、 1.2	两组测试合计 8 passed	—
2.1	`Generator`	无	3 个 `test_generator_` 测试	`src/models.py`
2.2	`Discriminator`	无	3 个 `test_discriminator_` 测试	`src/models.py`
2.3	`weights_init`	2.1、 2.2	2 个 `initialization` 测试	`src/models.py`
2	检查点 B： 网络测试通过	2.1～ 2.3	`uv run pytest tests/test_models.py`（9/9）	—
3.1	`discriminator_loss`	无	5 个判别器损失与标签测试	`src/losses.py`
3.2	`generator_loss`	无	1 个生成器损失测试	`src/losses.py`
3	检查点 C： 损失测试通过	3.1、 3.2	`uv run pytest tests/test_losses.py`（8/8）	—
4.1	`set_requires_grad`	无	1 个 `set_requires_grad` 测试	`src/training.py`
4.2	`discriminator_step`	1.1、 2、 3、4.1	1 个 `discriminator_step` 测试	`src/training.py`
4.3	`generator_step`	1.1、 2、 3、4.1	1 个 `generator_step` 测试	`src/training.py`
4.4	`train_one_epoch`	4.2、 4.3	2 个 `train_one_epoch` 测试	`src/training.py`
4	检查点 D： 训练测试通过	4.1～ 4.4	`uv run pytest tests/test_training.py`（10/10）	—
5.1	`batch_diversity`	无	3 个 `batch_diversity` 测试	`src/metrics.py`
5	检查点 E：全部 33 例通过	检查点 A～ D、 5.1	`uv run pytest`（33/33）	—
5.2	正式训练与对照实验	检查点 E	运行 `train_gan.py`，观察样本并记录指标	`train_gan.py`

Task 1：潜变量与插值

步骤 1.1：`sample_noise`

目标：从标准正态分布采样潜变量，作为生成器的输入。

函数签名：

```python
def sample_noise(batch_size, z_dim, device, generator=None):
    # 返回 (batch_size, z_dim) 的标准正态张量
```

实现要求：使用 `torch.randn`，直接在指定 `device` 上创建张量，并传入可选的 `torch.Generator` 以支持复现。`batch_size` 或 `z_dim` 非正时，抛出 `ValueError`。

验证：

```
uv run pytest tests/test_training.py -k sample_noise
```

应看到 5 passed。其中 2 个测试检查形状、设备、分布与随机种子复现，另外 3 个参数化用例检查非法尺寸。

步骤 1.2：`interpolate_latents`

目标：在两个潜变量之间做线性插值，用于观察生成图像随潜变量的变化。

$$z(t) = (1-t)z_0 + tz_1, \qquad t \in \left\{0, \frac{1}{K-1}, \ldots, 1\right\}, \quad K = \text{steps}.$$

函数签名：

```python
def interpolate_latents(z_start, z_end, steps):
    # 输入形状相同，返回 (steps, *z_start.shape) 的张量
```

实现要求：输出包含两端点，支持任意尾部形状，并保持输入的设备与数据类型。输入形状不同或 `steps < 2` 时，抛出 `ValueError`。

验证：

```
uv run pytest tests/test_metrics.py -k interpolation
```

应看到 3 passed，分别检查端点与中点、批量输入形状，以及非法输入。

检查点 A

完成上述两个函数后，运行：

```
uv run pytest tests/test_training.py tests/test_metrics.py -k "sample_noise or interpolation"
```

应看到 8 passed。此时只验证潜变量工具，训练与多样性相关测试将在 Task 4、Task 5 完成后验证。

Task 2：DCGAN 网络

步骤 2.1：`Generator`

目标：在 `src/models.py` 中实现从潜变量到图像的映射。设 `B=base_channels`，网络结构如下：

层	输入 → 输出	参数	归一化与激活
reshape	`(N,z_dim)` → `(N,z_dim,1,1)`	—	—

层	输入 → 输出	参数	归一化与激活
ConvTranspose2d	`(N,z_dim,1,1)` → `(N,4B,7,7)`	`k=7,s=1,p=0`	BatchNorm + ReLU
ConvTranspose2d	`(N,4B,7,7)` → `(N,2B,14,14)`	`k=4,s=2,p=1`	BatchNorm + ReLU
ConvTranspose2d	`(N,2B,14,14)` → `(N,1,28,28)`	`k=4,s=2,p=1`	Tanh

转置卷积的空间尺寸为（本实验 `dilation=1`）：

$$H_{out} = (H_{in} - 1)s - 2p + k + output_padding.$$

Tanh 将生成图像限制在 $[-1, 1]$，真实图像也必须归一化到相同范围，否则判别器可仅凭值域区分真假。数据加载代码已完成该处理。

验证：

```
uv run pytest tests/test_models.py -k test_generator_
```

应看到 3 passed，检查输出形状与值域、非法输入，以及转置卷积和 BatchNorm 的结构。

步骤 2.2：`Discriminator`

目标：为每张输入图像输出一个真假 logit。网络结构如下：

层	输入 → 输出	参数	归一化与激活
Conv2d	`(N,1,28,28)` → `(N,B,14,14)`	`k=4,s=2,p=1`	LeakyReLU(0.2)
Conv2d	`(N,B,14,14)` → `(N,2B,7,7)`	`k=4,s=2,p=1`	BatchNorm + LeakyReLU(0.2)
Conv2d	`(N,2B,7,7)` → `(N,1,1,1)`	`k=7,s=1,p=0`	无
reshape	`(N,1,1,1)` → `(N,)`	—	无

第一层不使用 BatchNorm，末层不使用 Sigmoid。注意保留 batch 维：即使 `N=1`，输出也应为 `(1,)`，而不是标量。

验证：

```
uv run pytest tests/test_models.py -k test_discriminator_
```

应看到 3 passed，检查每张图像对应一个 logit、网络中没有 Sigmoid，以及非法输入。

步骤 2.3：`weights_init`

目标：实现 DCGAN 的权重初始化规则：

- 卷积与转置卷积：$W \sim \mathcal{N}(0, 0.02^2)$，若有 bias 则置为 0；
- BatchNorm：$\gamma \sim \mathcal{N}(1, 0.02^2)$，$\beta = 0$。

通过 `apply` 递归初始化各层：

```
generator.apply(weights_init)
discriminator.apply(weights_init)
```

验证：

```
uv run pytest tests/test_models.py -k initialization
```

应看到 2 passed，分别检查卷积权重与 bias、BatchNorm 的增益与偏置。

检查点 B

完成两个网络与初始化函数后，运行：

```
uv run pytest tests/test_models.py
```

应看到 9 passed：上述 8 个测试加上初始就通过的构造参数检查。

Task 3：对抗损失

步骤 3.1：`discriminator_loss`

目标：在 `src/losses.py` 中实现真实样本与生成样本的二元交叉熵损失。

函数签名：

```
def discriminator_loss(real_logits, fake_logits, real_label=1.0):
    # 返回真实样本与生成样本两项平均 BCEWithLogits 损失之和
```

实现要求：真实样本的目标为 `real_label`，生成样本的目标为 0。分别调用 `binary_cross_entropy_with_logits`，再将两项相加；`real_label` 不在 `(0,1]` 内时，抛出 `ValueError`。

`real_label=0.9` 表示单边标签平滑[4]：只调低真实标签，不调高假标签。它可减轻判别器过度自信，但是否改善生成效果需要通过实验判断。

验证：

```
uv run pytest tests/test_losses.py -k "discriminator_loss or label"
```

应看到 5 passed，包括与 PyTorch 定义一致、单边标签平滑，以及 3 个非法标签用例。这组测试不依赖生成器损失，可以先单独验证。

步骤 3.2：`generator_loss`

目标：实现非饱和生成器损失，使生成图像被判为真。

函数签名：

```
def generator_loss(fake_logits):
    # 使用全 1 目标，返回平均 BCEWithLogits 损失
```

生成器的目标始终为 1，不随判别器的 `real_label` 改变。若误用目标 0，优化方向会变成帮助判别器识别假图。

验证：

```
uv run pytest tests/test_losses.py -k test_generator_loss
```

应看到 1 passed，检查结果是否与非饱和目标一致。

检查点 C

完成两个损失函数后，运行：

```
uv run pytest tests/test_losses.py
```

应看到 8 passed。除了上述 6 个测试，另有 2 个联合测试：`test_losses_are_finite_for_extreme_logits` 使用 ±1000 的分数检查数值稳定性；`test_losses_backpropagate` 检查梯度传播。这两个测试需在两个损失函数都完成后验证。

Task 4：交替训练与梯度隔离

每个 batch 依次更新 D 和 G，两个网络各使用一个优化器。更新 D 时将生成图像视为常量；更新 G 时冻结 D 的参数，但保留经过 D 回到 G 的梯度路径。

步骤 4.1：`set_requires_grad`

目标：统一开启或关闭网络参数的梯度计算。

遍历 `module.parameters()`，对每个参数调用 `parameter.requires_grad_(enabled)`。

验证：

```
uv run pytest tests/test_training.py -k set_requires_grad
```

应看到 1 passed，检查所有参数能否先关闭、再重新开启梯度。

步骤 4.2：`discriminator_step`

目标：只更新 D 的参数，不为 G 计算梯度。核心流程如下：

```
optimizer_d.zero_grad(set_to_none=True)
with torch.no_grad():
    fake_images = generator(z)
real_logits = discriminator(real_images)
fake_logits = discriminator(fake_images)
loss = discriminator_loss(real_logits, fake_logits, real_label)
loss.backward()
optimizer_d.step()
```

实现要求：

1. 将两个网络设为训练模式，并开启 D 参数的梯度计算。
2. 若未提供 `z`，按真实 batch 的大小与设备调用 `sample_noise`。
3. 生成假图时停止梯度，只调用 D 的优化器更新参数。
4. 返回包含 `d_loss`、`d_real`、`d_fake` 的字典，值均为 Python 浮点数。后两项分别是本次前向中真实、生成样本经 sigmoid 后的平均概率。

停止梯度可避免为 G 建立无用的计算图。这里的 sigmoid 仅用于记录指标，不参与损失计算。

验证：

```
uv run pytest tests/test_training.py -k discriminator_step
```

应看到 1 passed。该测试比较更新前后的参数，检查 D 参数改变、G 参数不变，以及本次操作没有为 G 产生梯度。

步骤 4.3：`generator_step`

目标：只更新 G 的参数，同时让梯度经过 D 回到 G。

实现要求：

1. 将两个网络设为训练模式，关闭 D 参数的梯度计算，并清空 G 优化器的梯度。
2. 若未提供 `z`，重新采样潜变量，计算 `G(z)`、`D(G(z))` 与生成器损失。
3. 反向传播并更新 G，随后恢复 D 参数的 `requires_grad=True`。
4. 返回 `(stats, fake_images.detach())`；`stats` 包含 Python 浮点数 `g_loss` 与 `g_score`，后者为本次前向中假图被判为真的平均概率。

不能用 `torch.no_grad()` 包住 D 的前向，否则会切断回到 G 的梯度。冻结参数与 `eval()` 也不同：`eval()` 只改变 BatchNorm / Dropout 的行为，并不关闭梯度。

验证：

```
uv run pytest tests/test_training.py -k generator_step
```

应看到 1 passed，检查 G 参数改变、D 参数不变、返回假图已脱离计算图，以及 D 的梯度开关最终被恢复。

步骤 4.4：`train_one_epoch`

目标：遍历数据，完成每批先 D 后 G 的交替更新，并汇总指标。

实现要求：

1. loader 返回 `(images, labels)` 时只取图像；直接返回图像张量时也应支持。将图像移到指定设备，GAN 不使用类别标签。
2. 每批依次调用 `discriminator_step` 和 `generator_step`，使用当前 batch 的实际大小。
3. 若指定 `max_batches`，达到限制后停止。按实际处理的 batch 数对五项指标求平均，返回含 `d_loss`、`g_loss`、`d_real`、`d_fake`、`g_score` 的字典。
4. 若未处理任何 batch，抛出 `ValueError`。

验证：

```
uv run pytest tests/test_training.py -k train_one_epoch
```

应看到 2 passed，分别检查完整交替更新与返回指标，以及空 loader 的处理。

检查点 D

完成所有训练工具后，运行：

```
uv run pytest tests/test_training.py
```

应看到 10 passed。该文件还包含 Task 1 的 5 个噪声采样测试，因此需要先完成 Task 1.1。参数隔离测试检查 D step 只改变 D 的参数，G step 只改变 G 的参数，并在结束后恢复 D 的梯度开关。

Task 5：训练、可视化与诊断

步骤 5.1：`batch_diversity`

目标：计算生成批次在样本维度上的逐像素标准差，再对所有像素取平均。

函数签名：

```
def batch_diversity(images):
    # images: (N,C,H,W), N >= 2; 返回标量张量
```

实现要求：使用 `unbiased=False`。例如，两张像素值分别为 0 和 2 的单像素图像应得到标准差 1；输入不是四维或样本数不足 2 时，抛出 `ValueError`。

该指标不能单独代表图像质量，但可辅助诊断模式崩塌：固定配置下，数值突然接近 0 往往意味着生成样本趋同。

验证：

```
uv run pytest tests/test_metrics.py -k batch_diversity
```

应看到 3 passed，检查相同图片的结果为 0、简单输入的手算值，以及样本数量不足的处理。随后运行本文件的全部测试：

```
uv run pytest tests/test_metrics.py
```

应看到 6 passed，其中包含 Task 1 的 3 个潜变量插值测试。

检查点 E：全部通过

完成所有待实现函数后，在项目根目录运行：

```
uv run pytest
```

应看到：

```
.................................                                        [100%]
33 passed
```

这是全部任务完成后的结果，与环境配置阶段的 `32 failed, 1 passed` 不同。若仍有失败，可按文件和测试名筛选，回到相应步骤检查。

随后运行不下载数据的流程检查：

```
uv run python train_gan.py --smoke --epochs 1 --max-batches 2 --batch-size 16 --base-channels 8
```

脚本应在 `outputs/` 中生成：

文件	内容
`samples_epoch_001.png`	固定噪声生成的样本网格
`loss_curve.png`	G / D 损失曲线
`latent_interpolation.png`	两个潜变量之间的插值图

文件	内容
`checkpoint.pt`	G / D 参数与命令行配置
`history.json`	逐 epoch 指标

> `--smoke` 使用随机图片，只检查训练与文件输出流程，不用于评价生成质量。

步骤 5.2：正式训练

通过测试与流程检查后，在 Fashion-MNIST 上训练。基线配置如下，训练流程可参考 PyTorch 的 DCGAN 教程[3]：

```
uv run python train_gan.py \
  --epochs 10 \
  --batch-size 128 \
  --z-dim 100 \
  --base-channels 32 \
  --lr 0.0002 \
  --beta1 0.5 \
  --real-label 1.0 \
  --seed 0
```

Fashion-MNIST 会在首次运行时自动下载。CPU 可完成训练，有 CUDA 时建议使用 GPU 加速。调试时可用 `--max-batches 100` 限制每个 epoch 的 batch 数；正式报告中须说明是否截断数据。

如何分析训练效果

GAN 的损失不是越低越好，应结合生成样本与跨 epoch 的指标变化判断：

现象	可能原因	需要结合的证据
D loss 很低，G loss 持续升高	判别器过强	D(real) 接近 1、D(fake) 接近 0，样本无改善
生成图像几乎相同	模式崩塌	固定噪声网格重复、多样性下降、插值变化很小
G / D loss 大幅振荡	学习率过高或双方失衡	连续 epoch 的样本质量反复退化
loss 稳定但图像质量差	尚未学到有效结构	样本网格、插值图与训练时长

每个 epoch 使用同一组固定噪声，才能排除输入变化、比较模型的变化。插值图则用于观察潜空间中相邻输入的生成结果是否连贯，是否出现大量重复或退化。

实验报告要求

使用 `report-template.md` 完成以下三组单变量对照。每组仅改变指定变量，其余配置保持一致，包括随机种子、epoch 数、batch size、通道数和数据范围。

对照实验	配置一	配置二
潜变量维度	`--z-dim 100`	`--z-dim 32`
Adam 动量	`--beta1 0.5`	`--beta1 0.9`
单边标签平滑	`--real-label 1.0`	`--real-label 0.9`

基线只需运行一次，可在三组比较中复用，因此共需完成四次训练。使用不同的 `--out-dir` 保存各配置的结果，避免覆盖。

每个配置需记录最终 D / G loss、D(real)、D(fake)、diversity，并附固定噪声样本网格与插值图。每组用几句话分析清晰度、多样性和训练稳定性，不要仅凭 loss 排名。

回答以下四个思考题：

1. 为什么判别器输出 logits，并使用 BCEWithLogits，而不是先计算 sigmoid 再计算 BCE?
2. 更新 D 时为什么要停止 G 的梯度？更新 G 时为什么仍需让梯度经过 D?
3. 为什么 GAN loss 不能单独反映生成质量?
4. 固定噪声、随机样本和潜变量插值分别揭示什么性质?

提供的文件

以下文件已经写好，直接使用：

文件	内容
`src/data.py`	Fashion-MNIST 下载、$[-1, 1]$ 归一化与 DataLoader
`train_gan.py`	命令行训练、设备选择及曲线、样本、插值、权重保存
`report-template.md`	报告结构与对照实验记录表
`tests/`	33 个无需下载数据的自动化测试

评分标准

本实验满分 100 分，由自动化测试与训练报告两部分组成。

第一部分：自动化测试

本部分满分 80 分，由 `uv run pytest` 共 33 例的通过比例决定：

$$\text{本部分得分} = \frac{\text{通过用例数}}{33} \times 80.$$

测试覆盖输出形状、网络结构、数值稳定性、梯度传播、参数隔离、异常输入与诊断函数。不得修改测试文件、针对测试值硬编码，或以其他方式绕过实际计算。

第二部分：训练与报告

本部分满分 20 分，评分要点如下：

1. 基线训练完整，产物可复现（4 分）；
2. 完成三组对照实验，控制变量公平（9 分）；
3. 对模式崩塌与训练稳定性的分析有图表和指标支持（4 分）；
4. 思考题回答正确，报告书写规范（3 分）。

提交要求

1. 运行 `uv run pytest` 并保存测试结果。

2. 提交源代码、实验报告、最终 `history.json`、`loss_curve.png`、一张最终样本网格和 `latent_interpolation.png`。
3. 删除 `.venv`、`.pytest_cache`、`data` 与大体积中间 checkpoint 后，将项目与报告打包为 `学号-姓名.zip`。
4. 仅修改 `src/` 内的待实现函数；如需扩展 `train_gan.py`，须在报告中说明。不得更改函数签名或约定的输入输出接口。

截止时间由任课教师另行通知。

参考资料

[1] Goodfellow, I., et al. (2014). *Generative Adversarial Nets*. NeurIPS. https://papers.neurips.cc/paper/5423-generative-adversarial-nets

[2] Radford, A., Metz, L., & Chintala, S. (2016). *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. ICLR. https://arxiv.org/abs/1511.06434

[3] PyTorch. *DCGAN Tutorial*. https://docs.pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html

[4] Salimans, T., et al. (2016). *Improved Techniques for Training GANs*. NeurIPS. https://papers.neurips.cc/paper/6125-improved-techniques-for-training-gans

[5] Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. https://arxiv.org/abs/1708.07747